

SOCIAL NETWORKS

NPTEL Online Course • Prof. Sudarshan Iyengar, IIT Ropar

Week 3 — Question Paper (Comprehension-Based MCQ / MSQ)

Comprehension 1 — The Strength of Weak Ties

Case Study

An employment researcher, Devika, studies how people actually land new jobs. Surveying a large alumni network, she keeps hearing the same surprising thing: the lead that produced the job rarely came from a close, everyday friend; far more often it came from a distant acquaintance someone the person barely keeps in touch with. To make sense of this she models the alumni as a graph: each person is a node, and a friendship is an edge that can be strong (frequent, close contact) or weak (occasional, distant contact).

She reasons about why a distant contact is so valuable. Close friends tend to move in the same circle, so they mostly know what you already know; a distant contact lives in a different cluster and carries information you would not otherwise reach. She also revisits the local structure of friendships: when one person is close to two others, those two tend, eventually, to meet and befriend each other. And she pays attention to edges that join two otherwise-separate clusters, since pulling such an edge would cut the network in two.

To make these ideas concrete she keeps a few small examples in front of her. In one, a person P has five close friends and, among those five, exactly four friendships exist. In another, she looks at a single friendship between two people X and Y: X's friends are {C, D, E} and Y's friends are {C, D, F}. She is also interested in how a person who is the sole connection between two communities can quietly profit from that position.

Questions 1–10

Q1. [MCQ] In Devika's survey, most job leads come through weak ties (distant acquaintances) rather than strong ties (close friends). What is the best explanation?

- A. Close friends deliberately keep job openings to themselves, so leads arrive only through acquaintances.
- B. Weak ties reach into a different circle, so they carry news your close friends don't already have.
- C. Weak ties simply outnumber strong ties, so by sheer chance more leads happen to pass through them.
- D. Strong ties cannot transmit information at all, leaving only weak ties able to pass job news along.

Answer: B

Reference: Lecture 28 – Granovetter's Strength of Weak Ties (≈01:34–02:32)

Solution: Close friends share your circle and largely know what you already know; a weak tie reaches a different world and brings genuinely new information, such as an unfamiliar opening. It is this novelty — not secrecy, not sheer numbers, and not any inability of strong ties to communicate — that makes weak ties so useful in a job search.

Q2. [Match the following] Match each concept in Column A with its correct definition in Column B.

Column A (concept)	Column B (definition)
1. Triadic closure	a. an edge whose two endpoints have no common friend (no triad on it)
2. Local bridge	b. the number of common friends shared by the two endpoints of an edge
3. Embeddedness	c. two people with a common friend becoming friends with each other
4. Neighbourhood overlap	d. (common friends) ÷ (all distinct friends) of an edge's two endpoints

- A. 1-a, 2-c, 3-d, 4-b
- B. 1-c, 2-a, 3-b, 4-d
- C. 1-c, 2-b, 3-a, 4-d
- D. 1-d, 2-a, 3-b, 4-c

Answer: B (1-c, 2-a, 3-b, 4-d)

Reference: Lecture 29 – Triads, clustering coefficient and neighbourhood overlap; Lecture 30 – bridges; Lecture 32 – embeddedness

Solution: Triadic closure is two friends-of-a-friend becoming friends (1-c). A local bridge is an edge with no triad on it — its endpoints share no common friend (2-a). Embeddedness of an edge is the count of common friends of its endpoints (3-b). Neighbourhood overlap normalises that count by the union of the two friend sets (4-d).

Q3. [MCQ] Person P has five close friends, and among those five there are exactly four friendships. What is P's clustering coefficient?

- A. $4/5 = 0.80$
- B. $4/25 = 0.16$
- C. $4/10 = 0.40$
- D. $5/10 = 0.50$

Answer: C

Reference: Lecture 29 – Triads, clustering coefficient and neighbourhood overlap (≈03:23–04:29)

Solution: Clustering coefficient = (friendships among your friends) ÷ (possible friendships among them). With 5 friends, the possible friendships number $C(5,2) = 10$, and 4 exist, so the coefficient is $4/10 = 0.40$. A value near 1 means your friends all know each other; near 0 means they don't.

Q4. [MCQ] For the friendship between X and Y, X's friends are {C, D, E} and Y's friends are {C, D, F}. What is the neighbourhood overlap of the edge X–Y?

- A. $2/3 \approx 0.67$
- B. $2/6 \approx 0.33$
- C. $3/3 = 1.00$

D. $2/4 = 0.50$

Answer: D

Reference: Lecture 29 – Triads, clustering coefficient and neighbourhood overlap (≈06:14–08:54)

Solution: Neighbourhood overlap = $| \text{common friends} | \div | \text{all distinct friends} |$. Common = {C, D} (2); union = {C, D, E, F} (4). So the overlap is $2/4 = 0.50$. A small overlap signals an edge close to a local bridge (a weak tie); a large overlap signals a strongly embedded, strong tie.

Q5. [MCQ] Using the same two friend sets {C, D, E} for X and {C, D, F} for Y, what is the embeddedness of the edge X–Y?

- A. 2
- B. 4
- C. 6
- D. 0

Answer: A

Reference: Lecture 32 – Embeddedness (≈01:03)

Solution: Embeddedness of an edge is simply the number of common friends of its endpoints — here {C, D}, so 2. (Neighbourhood overlap in the previous question is this same count divided by the size of the union.) Higher embeddedness is associated with more trust between the two people.

Q6. [MCQ] Devika notes the relationship between bridges and local bridges. Which statement is correct?

- A. Every bridge is a local bridge, but not every local bridge is a bridge.
- B. Every local bridge is a bridge, but not every bridge is a local bridge.
- C. A bridge and a local bridge are simply two names for the same thing.
- D. Neither a bridge nor a local bridge can ever turn out to be a weak tie.

Answer: A

Reference: Lecture 30 – Structure of weak ties, bridges, and local bridges (≈02:50–03:29)

Solution: A bridge is an edge whose removal disconnects the graph; a local bridge is the weaker notion of an edge with no triad on it (its endpoints share no common friend). Every bridge necessarily has no common-friend triad, so every bridge is a local bridge; but a local bridge need not disconnect the graph, so the converse fails. Both tend to be weak ties.

Q7. [MCQ] Granovetter argued that a local bridge is almost always a weak tie. Using the strong-triadic-closure property, which reasoning supports this?

- A. Two people in separate clusters rarely meet, so any cross-cluster edge must be weak, regardless of closure.
- B. A local bridge has to be a strong tie, since only a strong tie can reach across two separate clusters.
- C. If A–B were strong and A also had a strong tie Z, closure forces Z–B, so A and B share a friend.
- D. Removing any strong tie disconnects the graph, so every strong tie is by definition a bridge and a local bridge.

Answer: C

Reference: Lecture 30 – Structure of weak ties, bridges, and local bridges (≈11:09)

Solution: Under strong triadic closure, a person's two strong ties become connected. So if the local bridge A–B were strong and A had any other strong tie Z, then Z–B would form — creating a common friend of A and B and destroying the “no triad” property. Hence a local bridge can't be a strong tie; it is a weak tie. The other options misstate the bridge/strength relationship.

Q8. [MCQ] Sudarshan is the only person linking his community to Ramya, who sits in a separate community of real-estate agents. What does this structure illustrate, and who benefits?

- A. A structural hole; Ramya benefits, since every contact to her community must pass through her (brokerage).
- B. High embeddedness; the two communities share many friends, so both sides benefit equally from the link.
- C. Triadic closure; the two separate communities will soon merge into one single large community.
- D. A clustering coefficient of one; the single link makes the entire network completely connected.

Answer: A

Reference: Lecture 33 – Structural Holes (≈00:51–end)

Solution: The gap between the two communities, spanned by a single low-embeddedness edge, is a structural hole. Ramya brokers it: anyone on Sudarshan's side who needs the other community must go through her, so she profits from the monopoly position. This is the network-structure flip-side of Granovetter's weak ties; it is not high embeddedness, triadic closure, or full connectivity.

Q9. [MSQ] Which statements about embeddedness and trust are correct? (Select all that apply.)

- I. Embeddedness is a property of an edge (a relationship), not of a single node.
 - II. Higher embeddedness (more common friends) is generally associated with greater trust between the two people.
 - III. A low-embeddedness edge can still be valuable, because it may give one endpoint a brokerage position between communities.
 - IV. Embeddedness equals the total number of friends a single person has.
- A. I, II, IV
 - B. I, II, III
 - C. II, III, IV
 - D. I, III

Answer: B (statements I, II, III)

Reference: Lecture 32 – Embeddedness; Lecture 33 – Structural Holes

Solution: Embeddedness is defined per edge (the common friends of its two endpoints), and more common friends usually means more trust (statements I and II). A low-embeddedness edge can still confer a brokerage advantage across a structural hole (III). Counting a single person's friends describes that node's degree, not an edge's embeddedness, so IV is false.

Q10. [MSQ] Which statements about social capital, closure, and brokerage are correct? (Select all that apply.)

- I. Closure refers to friends-of-friends becoming friends, which raises clustering and trust.
 - II. Brokerage refers to spanning a structural hole between otherwise-separate groups.
 - III. A network where everyone is friends with everyone (100% closure) maximises healthy competition between distinct groups.
 - IV. Both closure and brokerage can be valuable; the ideal social structure is an open research question.
- A. I, II, III
 - B. II, III, IV
 - C. I, II, IV
 - D. I, IV

Answer: C (statements I, II, IV)

Reference: Lecture 34 – Social Capital (closure vs brokerage)

Solution: Closure (friends-of-friends linking) and brokerage (spanning a hole) are the two mechanisms, and both matter — the optimal mix is unresolved (statements I, II, IV). Total closure leaves no distinct groups to compete (the “everyone-on-one-team” case the lecture calls boring), so III is false: competition needs separate communities linked by brokers.

Comprehension 2 — Finding the Hidden Groups: Community Detection

Case Study

A platform-integrity team wants to split a user-interaction graph into communities tightly knit groups with many links inside and few links across. Visualising the graph rarely reveals the groups directly, so the team turns to two algorithms covered in the course.

The first is a brute-force partitioner. It enumerates every way of splitting the nodes into two parts, and for each split scores how community-like it is by comparing the edges that stay inside the two parts against the edges that cross between them; the split with the best score wins. To build the splits it relies on Python's `itertools.combinations`, and to count the edges that fall inside a candidate part it uses an induced subgraph. For a tiny test it uses a barbell graph two cliques of five joined by a single connecting edge, whose obvious communities are nodes `{0,1,2,3,4}` and `{5,6,7,8,9}`.

The second is the Girvan–Newman algorithm. Its insight is that the edges bridging two communities act like highways: an unusually large share of the network's shortest paths must funnel through them, so those bridging edges carry a high value of edge betweenness. The algorithm repeatedly measures edge betweenness, deletes the most “highway-like” edge, and watches for the graph to fall apart into separate pieces those pieces are the communities. It is far faster than brute force on large graphs.

```
import networkx as nx
from itertools import combinations

# brute force: score a split by intra vs inter edges
sub = G.subgraph(first_community)      # induced subgraph
intra1 = sub.number_of_edges()         # edges inside part 1
inter = G.number_of_edges() - intra1 - intra2
ratio = (intra1 + intra2) / inter      # higher = better split
```

Questions 1–10

Q1. [MCQ] How does the team define a good community partition of the user graph?

- A. Every node in the graph has exactly the same number of links as every other.
- B. Each group forms a complete subgraph in which all of its members are linked.
- C. The two groups always contain exactly the same number of nodes as each other.
- D. Each group has many links inside it and only few links to the other groups.

Answer: D

Reference: Lecture 35 / 39 – community definition

Solution: A community is a set of nodes densely linked within and sparsely linked to the rest — high intra-community edge density, low inter-community density. It need not be a clique, need not be equal-sized, and does not require every node to have the same degree.

Q2. [MCQ] The brute-force code builds candidate parts with `itertools.combinations`. What does `list(combinations([1,2,3,4], 2))` produce?

```
from itertools import combinations
list(combinations([1, 2, 3, 4], 2))
```

- A. All twelve ordered pairs, counting (1,2) and (2,1) as two different pairs.
- B. All six unordered 2-element subsets of the list, each one listed exactly once.
- C. The four single-element tuples (1,), (2,), (3,), and (4,) on their own.
- D. One single tuple that contains all four elements together, (1, 2, 3, 4).

Answer: B

Reference: Lecture 35 – Finding Communities, Brute Force (≈03:39)

Solution: `combinations(list, 2)` returns every unordered 2-element combination as a tuple — $C(4,2) = 6$ of them: (1,2),(1,3),(1,4),(2,3),(2,4),(3,4). Order is ignored, so (2,1) never appears (that would be permutations). The result is an iterator; wrapping it in `list()` materialises the six tuples.

Q3. [MCQ] Inside the loop the code calls `G.subgraph(first_community).number_of_edges()`. What does this value represent?

- A. The number of edges joining nodes that both lie within `first_community` only.
- B. The total number of edges present in the whole graph `G`, across both groups.
- C. The number of edges that cross between the first and the second community.
- D. The number of nodes that are contained in the set `first_community`.

Answer: A

Reference: Lecture 35 – Finding Communities, Brute Force (≈14:05–18:31)

Solution: `G.subgraph(nodes)` returns the induced subgraph on those nodes — it keeps only edges whose both endpoints lie in the set. So `number_of_edges()` on it counts the intra-community edges of that part. Inter-community edges are then obtained by subtracting both parts' intra counts from the graph's total.

Q4. [MCQ] For one candidate split the team finds 10 intra-edges in part 1, 10 intra-edges in part 2, and 2 edges crossing between the parts. Using $\text{ratio} = (\text{intra1} + \text{intra2}) / \text{inter}$, what is the score, and how is it used?

- A. 0.1, and the split with the lowest ratio is chosen as best.
- B. 2, since the ratio just equals the number of crossing edges.
- C. 10, and the split with the highest ratio is chosen as best.
- D. 22, since the ratio counts every edge in the whole graph.

Answer: C

Reference: Lecture 35 – Finding Communities, Brute Force (≈13:49–23:15)

Solution: $\text{ratio} = (10 + 10) / 2 = 10$. A high ratio means dense connections inside the groups and sparse connections between them, so the brute-force method keeps the division with the maximum ratio (found via `max()` and its index) as the best partition.

Q5. [MCQ] Why does the team consider the brute-force method impractical for large graphs?

- A. It only runs on graphs that have fewer than ten edges in total.
- B. The number of two-way splits grows like $2^{(n-1)}$, which explodes as n gets large.
- C. It has no way to count the number of edges inside a candidate subgraph.
- D. It always returns the wrong communities, no matter how small the graph happens to be.

Answer: B

Reference: Lecture 39 – Betweenness Measures and Graph Partitioning (≈04:02); Lecture 36

Solution: Brute force checks essentially every split of the node set; the count grows exponentially (on the order of $2^{(n-1)}$), so it becomes infeasible for large graphs — the lecture notes around 2^n cases for $n = 50$. For $n = 10$ that is already $2^9 = 512$ divisions. Girvan–Newman avoids enumerating splits.

Q6. [MCQ] What is the central idea that lets Girvan–Newman find communities by edge betweenness?

- A. Bridging edges carry high betweenness, since most cross-community shortest paths run through them.
- B. Edges that lie deep inside a single community carry the highest betweenness values of all.
- C. Every edge in the graph has an identical betweenness, so any edge may be removed first.
- D. Removing the lowest-betweenness edge first is what reveals the community structure.

Answer: A

Reference: Lecture 36 – Girvan–Newman (≈00:30); Lecture 39 (≈09:02–13:19)

Solution: Inter-community (bridging) edges are bottlenecks: nearly all shortest paths between the two groups funnel through them, giving them high edge betweenness. Removing the highest-betweenness edges first cuts these bridges and exposes the communities. Intra-community edges have many alternative paths around them, so they carry lower betweenness.

Q7. [Arrange in order] Arrange the steps of one round of the Girvan–Newman algorithm in the correct order.

Steps (in scrambled order):

- (i) Remove the edge with the highest betweenness from the graph.
 - (ii) Compute the edge betweenness centrality of every edge.
 - (iii) Check whether the graph has split into more connected components.
 - (iv) If still connected, repeat; once it splits, return the connected components as communities.
- A. i → ii → iii → iv
 - B. ii → i → iii → iv
 - C. iii → ii → i → iv
 - D. ii → iii → i → iv

Answer: B (ii → i → iii → iv)

Reference: Lecture 36 – Girvan–Newman (≈01:55–12:45)

Solution: Each round first computes edge betweenness for all edges (ii), then removes the single highest-betweenness edge (i), then tests connectivity (iii); while the graph stays connected the loop repeats, and as soon as it disconnects the components are returned as communities (iv).

Q8. [MCQ] In the implementation, `nx.edge_betweenness centrality(G)` is called and its result is processed as below. What does the function return, and what does the sort achieve?

```
dict1 = nx.edge_betweenness centrality(G)
lot = list(dict1.items())
lot.sort(key=lambda x: x[1], reverse=True)
return lot[0][0]    # the edge to remove
```

- A. A list of nodes; the sort just arranges those nodes into alphabetical order.
- B. A single betweenness number; the sort applied to one value has no real effect.
- C. A dict {edge: betweenness}; the sort puts the largest first, so `lot[0][0]` is the edge to drop.
- D. The shortest path between the two endpoints; the sort simply reverses the order of that path.

Answer: C

Reference: Lecture 36 – Girvan–Newman (≈04:00–08:00)

Solution: `edge_betweenness centrality` returns a dictionary mapping each edge to its betweenness. Converting with `.items()` gives (edge, value) tuples; sorting with `key=lambda x: x[1], reverse=True` orders them by betweenness, largest first. `lot[0]` is that top tuple and `lot[0][0]` is its edge — the one removed (unpacked with `*` into `G.remove_edge()`).

Q9. [MCQ] Which description best matches the definition of edge betweenness used to drive the partitioning?

- A. The number of neighbours that the edge's two endpoints happen to share.
- B. The total number of edges present in the graph, minus exactly one.
- C. The length of the longest shortest-path found anywhere in the graph.
- D. Over all node pairs s, t , the fraction of shortest $s-t$ paths that use the edge.

Answer: D

Reference: Lecture 39 – Betweenness Measures and Graph Partitioning (≈11:35)

Solution: Edge betweenness sums, over every pair of nodes s and t , the fraction $\sigma_{st}(e)/\sigma_{st}$ — the share of shortest $s-t$ paths that run through edge e . Bridging edges score highly because they lie on a large fraction of cross-community shortest paths. It is not a shared-neighbour count, an edge total, or a longest-path length.

Q10. [MSQ] Comparing the two methods on the barbell test graph and in general, which statements are correct? (Select all that apply.)

- I. On the barbell graph both methods recover the communities $\{0,1,2,3,4\}$ and $\{5,6,7,8,9\}$.
 - II. Girvan–Newman is generally much faster than brute force on large graphs.
 - III. `nx.connected_component_subgraphs(G)` (used in Girvan–Newman) returns the connected components as subgraphs, so its length tells you how many components there are.
 - IV. Brute force is preferred for large graphs because it always runs faster than Girvan–Newman.
- A. I, II, IV
 - B. II, III, IV
 - C. I, II, III
 - D. I, III

Answer: C (statements I, II, III)

Reference: Lecture 35 & 36 – Brute Force and Girvan–Newman

Solution: Both methods split the barbell into its two cliques (I); Girvan–Newman avoids enumerating all splits and is far faster at scale (II); and `connected_component_subgraphs` returns the components as graph objects, so its length is the component count used to detect the split (III). Brute force is the slow, exponential method, not the fast one, so IV is false.